

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	파비스	팀 인 원 (팀장 포함)	1명
참가부문	학생	과제유형	자유과제

□ 결과보고서

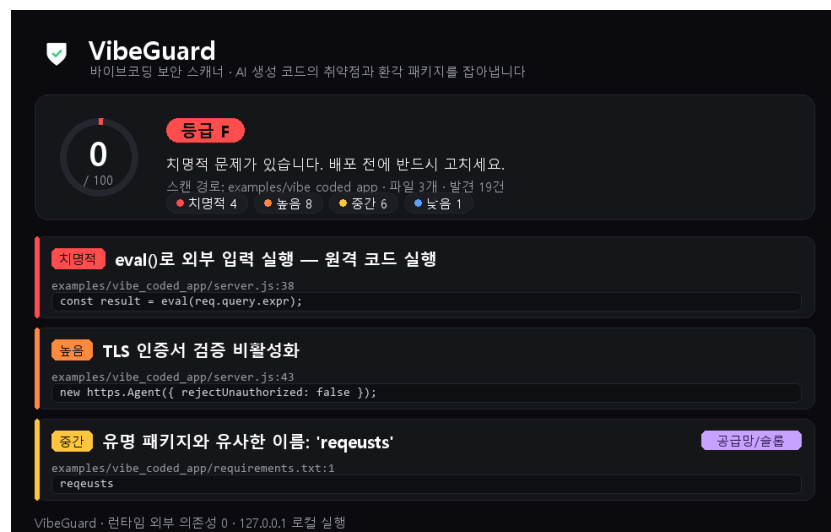
프로젝트 개요	
프로젝트명	VibeGuard 바이브코딩 보안 스캐너 ※ 수상 시 상장에 그대로 기재되므로, 특수문자를 제외한 핵심 키워드 중심의 프로젝트명 기재
프로젝트 등록 URL	https://github.com/nohseongmin/VibeGuard
시연영상	(제출 전 시연영상 유튜브 URL 기재 예정)
프로젝트 소개	코딩을 몰라도 실행파일을 받아 검사할 폴더만 끌어다 놓으면 된다. AI 코딩 도구가 만든 코드의 보안 취약점과 AI가 지어낸 가짜 패키지(슬롭스쿼팅)를 자동으로 찾아, 심각한 순서대로 어디가 왜 위험하고 어떻게 고치는지 보여 주는 오픈소스 보안 스캐너다.
프로젝트 세부 내용	
개발배경 및 목적	이제 비전문가도 AI에게 코드를 맡겨 앱·서비스를 만드는 '바이브코딩'이 보편화됐지만, 생성된 코드에는 하드코딩된 비밀번호, eval 등 위험 함수, 취약한 암호, SQL 명령 인젝션 같은 보안 결함이 그대로 포함되는 경우가 많다. 특히 거대언어모델이 존재하지 않는 패키지명을 환각으로 추천하고 공격자가 그 이름을 악성 패키지로 선점하는 슬롭스쿼팅(slopsquatting) 공급망 위협도 새롭게 대두되고 있다. 그러나 보안은 비전문가에게 너무 어렵다. 개발 목적은 코딩을 몰라도 커밋·배포 전에 코드의 취약점과 환각 패키지를 자동으로 점검받고 무엇이 왜 위험하며 어떻게 고치는지 쉬운 말로 안내받아, 평범한 사람들의 창의적인 아이디어가 안전하게 세상에 나와 상용화되도록 돕는 것이다.

개발환경	- 언어: Python 3.12 (런타임 외부 의존성 없이 표준 라이브러리만 사용) - 실행 환경: Windows / macOS / Linux 크로스플랫폼 명령행(CLI) 도구 - 테스트: pytest (단위 테스트 91건) - 패키징: pyproject.toml 기반 콘솔 스크립트(vibeguard) - 개발 도구: Visual Studio Code, Git, GitHub, Claude Code(코딩 보조)
시스템 구성 및 아키텍처	- 입력: 사용자가 지정한 파일·폴더의 소스코드 - CLI(cli.py): 명령 파싱 및 실행(scan / rules / init-hooks) - 스캐너(scanner.py): 대상 파일 수집 후 규칙 엔진 적용 - 규칙 엔진(rules/): secrets-dangerous-injection-web-crypto 5개 규칙군이 정규식·패턴 매칭으로 취약점 탐지 - 슬롭스쿼팅 검사(slopsquat.py): 패키지명을 인기 패키지명과 편집거리로 비교하고 PyPI·npm 레지스트리 실존 여부 조회 - 점수화(score.py): 심각도별 집계 후 0~100 보안 점수·등급 산정 - 리포터(reporter.py): 터미널(한국어) 또는 JSON 형식 출력 - git pre-commit 훅(hooks/): 커밋 시 자동 스캔으로 위험 코드 차단 - 데이터 흐름: 소스코드 → 규칙별 Finding(심각도·위치·CWE·해결책) 수집 → 점수 집계 → 리포트
프로젝트 주요기능	프로젝트 상세 내용 - 6개 영역 취약점 탐지: 하드코딩 비밀키·토큰, eval·exec 등 위험 실행, SQL·명령 인젝션, 안전하지 않은 웹 설정(0.0.0.0 바인딩, TLS 검증 비활성화), 취약 암호(MD5-SHA1, 안전하지 않은 난수) 등 (각 발견에 매핑) - 슬롭스쿼팅·오타스쿼팅 탐지: 인기 패키지명과 편집거리 분석과 레지스트리 실존 조회로 환각·오타 패키지를 경고하는 차별화 기능 - 보안 점수(0~100)와 등급(A~F) 산정 및 심각도별 집계 - 한국어 리포트: 발견마다 위치·코드·설명·해결책·CWE 제시(터미널·JSON 출력 지원) - git pre-commit 훅 자동 설치(init-hooks)와 --fail-on 옵션으로 커밋·CI 단계 차단 - 개발 과정: 규칙 엔진 → 슬롭스쿼팅 탐지 → CLI·리포터·훅 → 데모 취약 앱·단위 테스트 순으로 구현, 전 규칙 테스트 통과 - 브라우저 GUI(vibeguard gui): 외부 의존성 없이 로컬 웹 UI로 보안 점수·취약점 카드·슬롭스쿼팅을 시각화 - 다양한 출력과 연동: 터미널·JSON·Markdown·SARIF(GitHub 코드 스캐닝)·단독 HTML 리포트, GitHub Actions CI(SARIF 업로드), 베이스라인(기존 코드 수용)·설정 파일(.vibeguard.json) 지원 - 총 54개 규칙(시크릿·위험실행·인젝션·웹·암호·경로·공급망, 지원 언어

Python·JS·TS·Go·PHP·Ruby·Java)과 단위 테스트 91개 통과, 도구 자체 코드는 발견 0건

구동 및 시연

- 설치: 저장소 클론 후 `pip install -e .` (또는 `python -m vibeguard` 로 즉시 실행)
- 사용: `vibeguard scan <경로> / vibeguard scan . --format json / vibeguard init-hooks`
- 데모: `examples/vibe_coded_app`(의도적으로 취약하게 작성한 Flask·Express 예제 앱) 스캔 시 취약점 22건 탐지(치명 4·높음 11·중간 6·낮음 1), 슬롭스쿼팅 `expres-reqeusts` 포함, 보안 점수 0/100(F등급)
- 테스트: `pytest` 단위 테스트 91건 전부 통과
- GUI 실행: `vibeguard gui` → 브라우저에서 경로 입력·스캔(보안 점수 링·취약점 카드·슬롭스쿼팅 확인)
- 비개발자 실행: GitHub Releases에서 단일 실행파일(Windows·macOS·Linux)을 받아 검사할 폴더를 끌어다 놓으면 브라우저에 리포트가 뜬(Python·터미널 불필요). 더블클릭 런처(`VibeGuard-GUI.bat/.command`)도 제공



[그림] VibeGuard 브라우저 GUI 스캔 결과 화면

※ 필요시 도식, 사진 등 시각 자료 활용

기대효과 및 활용분야	향후 확장성 및 기대효과 - 비전문가·학생·1인 개발자가 AI 생성 코드를 안전하게 사용하도록 진입장벽을 낮춘 보안 점검 제공 - CI/CD 파이프라인, git 혹은, IDE 연동으로 개발 전 과정의 가드레일로 확장 가능 - 규칙 플러그인 구조로 지원 언어·프레임워크 확대가 용이하고, 슬롭스쿼팅 사전 확장으로 공급망 보안 강화 - 시큐어코딩 교육 교보재, 사내 코드리뷰 보조 등으로 활용 가능 - 생성형 AI 코딩 시장 성장에 따라 'AI 생성 코드 보안 점검' 수요 확대 기대 - 코딩을 모르는 사람도 보안 걱정 없이 아이디어를 서비스로 만들 수 있어, 보안이 막연해 멈췄던 1인 개발자·학생·스타트업의 창의적 아이디어가 더 안전하게 상용화될 수 있음
기타	프로젝트의 혁신성 및 차별성 기존 정적 분석·시크릿 스캐너와 달리 '바이브코딩' 시나리오에 특화되어 있다. 특히 거대언어모델의 환각 패키지(슬롭스쿼팅)를 편집거리와 레지스트리 조회로 탐지하는 기능은 일반 린터에서 보기 드문 독창적 강점이다. 또한 런타임 외부 의존성이 전혀 없어 보안 도구 자체의 공급망 위험이 0이며, 한국어 친화 리포트와 git 혹은 원클릭 연동으로 초심자도 즉시 사용할 수 있다. 또한 Python AST 분석으로 문자열·문서 안의 가짜 양성을 걸러 정밀도를 높였다. 한계점 및 향후 발전 로드맵 현재는 정규식·휴리스틱 기반이라 일부 오탐·미탐이 존재한다. 향후 추상구문트리(AST) 기반 정밀 분석을 강화하고, 지원 언어를 확대(현재 Python·JavaScript 중심)하며, 슬롭스쿼팅 인기 패키지 사전을 자동 갱신할 계획이다. 또한 IDE 플러그인과 SARIF 출력, 자동 수정(fix) 제안 기능을 추가해 실무 도입성을 높이려 한다. 소감 및 후기 AI가 만든 코드를 점검하는 도구를 만들며 생성형 AI 시대의 새로운 공급망 위험을 직접 구현하고 탐지해 본 점이 의미 있었다. 외부 의존성 없이 표준 라이브러리만으로 가볍게 동작하도록 설계하면서 규칙 엔진 구조와 단위 테스트의 중요성을 체감했다.

붙임1

SBOM(소프트웨어 자재명세서)

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	pytest	7.0 이상	MIT	https://github.com/pytest-dev/pytest	단위 테스트 실행(개발·테스트용 의존성). 런타임 외부 의존성은 없음(Python 표준 라이브러리만 사용).
2					
3					
4					
5					

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ☐ 표시)

- ☐ 유형 1: 외부 모델 그대로 활용 (추가 학습 없이 기존 공개 모델을 프로젝트에 연동·구동한 경우)
- ☐ 유형 2: 외부 모델 파인튜닝 (기존 공개 모델을 가져와 준비한 데이터셋으로 추가 미세조정한 경우)
- ☐ 유형 3: 자체 개발 모델 (기반 모델 없이 참가팀이 처음부터 가중치를 직접 전체 학습시킨 경우)
- ※ 개발 과정에서 코딩 및 디버깅 보조용으로 상용 AI(GPT, Claude 등)를 단순 활용한 경우는 체크하지 않음(4번 항목에 기재)

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	해당 없음 (AI 모델 미탑재)	기반 모델 라이선스	해당 없음
--------------	-------------------	------------	-------

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	해당 없음
데이터 정제/ 가공 방법 요약	해당 없음
새로 생성된 가중치 공개 저장소 URL	해당 없음
가중치 파일 정보 및 배포방식	해당 없음

4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)

직접 작성한 코드의 오픈소스 라이선스	MIT License	학습/추론 소스코드 공개 저장소 URL	https://github.com/nohseongmin/VibeGuard
상용 AI 보조도구 활용 여부 및 범위	코드 작성·디버깅 및 문서화 보조용으로 Anthropic Claude(Claude Code)를 활용함. 모든 산출물은 팀이 직접 검토·수정함.		